



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por nombre alumno
en Universidad de Burgos — 1 de junio
de 2026

Tutor: nombre tutor

Índice general

Índice de figuras

Índice de tablas

Apéndice A

Plan de Proyecto Software

A.1. Introducción

El presente anexo recoge los aspectos relacionados con la planificación y viabilidad del proyecto desarrollado. Se describen las principales fases de trabajo realizadas durante el desarrollo del sistema SIEM, así como una estimación de los costes asociados y los aspectos legales considerados.

Aunque el proyecto se ha desarrollado con fines académicos, resulta interesante analizar su viabilidad desde una perspectiva similar a la utilizada en proyectos profesionales de desarrollo software.

A.2. Planificación temporal

El desarrollo del proyecto se ha realizado siguiendo una metodología incremental, avanzando progresivamente desde los componentes más básicos hasta la integración completa del sistema.

Las principales fases del proyecto han sido las siguientes:

- Análisis inicial de requisitos y definición de objetivos.
- Diseño de la arquitectura general del sistema.
- Diseño del entorno simulado del polvorín.
- Definición del formato de eventos y logs.
- Desarrollo del módulo de ingesta.

- Implementación de la base de datos SQLite.
- Desarrollo del motor de correlación.
- Implementación del módulo de detección de anomalías.
- Desarrollo de la API REST mediante FastAPI.
- Desarrollo del dashboard de monitorización mediante Streamlit.
- Pruebas, validación y documentación final.

La naturaleza incremental del proyecto permitió validar cada módulo de forma independiente antes de proceder a su integración con el resto de componentes.

A.3. Estudio de viabilidad

Antes de iniciar el desarrollo del proyecto se realizó un análisis preliminar para determinar si la solución propuesta era viable desde el punto de vista técnico, económico y legal.

El objetivo de este estudio fue comprobar que las tecnologías seleccionadas permitían desarrollar un sistema SIEM funcional dentro de las limitaciones temporales y de recursos propias de un Trabajo Fin de Grado.

Tras analizar las herramientas disponibles y los requisitos planteados, se concluyó que el proyecto era viable utilizando tecnologías de código abierto, una arquitectura ligera y un entorno simulado que permitiera validar el funcionamiento del sistema sin necesidad de disponer de infraestructuras reales.

A continuación se analizan los aspectos económicos y legales más relevantes relacionados con el desarrollo del proyecto.

Viabilidad económica

Desde el punto de vista económico, el proyecto presenta una elevada viabilidad debido al uso de tecnologías de código abierto y herramientas gratuitas.

Las principales tecnologías utilizadas durante el desarrollo han sido Python, SQLite, FastAPI, Streamlit, Git y GitHub, todas ellas disponibles sin costes de licencia.

La estimación económica más relevante corresponde al tiempo de desarrollo invertido. Considerando un escenario profesional y una dedicación aproximada de 300 horas de trabajo a un coste estimado de 20 euros por hora, el coste de desarrollo ascendería aproximadamente a 6.000 euros.

No obstante, al tratarse de un proyecto académico desarrollado con herramientas gratuitas y sin necesidad de infraestructuras específicas, los costes reales han sido mínimos.

Viabilidad legal

El proyecto ha sido desarrollado utilizando exclusivamente tecnologías de código abierto o herramientas con licencias gratuitas para uso académico.

El entorno utilizado corresponde a una simulación ficticia de un polvorín militar y no incorpora información sensible, procedimientos operativos reales ni documentación clasificada.

Asimismo, se han tenido en consideración principios generales relacionados con la monitorización de eventos y la protección de infraestructuras críticas, evitando en todo momento la inclusión de información que pudiera comprometer la seguridad de instalaciones reales.

Por todo ello, no se identifican impedimentos legales para el desarrollo y utilización académica del sistema presentado en este Trabajo Fin de Grado.

Apéndice B

Especificación de Requisitos

B.1. Introducción

En este anexo se recogen los requisitos funcionales y no funcionales definidos para el desarrollo del sistema SIEM. Asimismo, se describen los principales casos de uso que permiten comprender las funcionalidades implementadas y la interacción del usuario con el sistema.

La especificación de requisitos ha servido como guía durante todo el proceso de desarrollo, permitiendo verificar posteriormente que los objetivos planteados inicialmente han sido alcanzados.

Una muestra de cómo podría ser una tabla de casos de uso:

B.2. Objetivos generales

Los objetivos generales definidos para el sistema son los siguientes:

- Centralizar la información procedente de diferentes dispositivos de seguridad.
- Almacenar de forma persistente los eventos generados por el entorno monitorizado.
- Detectar situaciones potencialmente peligrosas mediante reglas de correlación.
- Identificar anomalías relacionadas con accesos, sensores y condiciones ambientales.

- Facilitar la consulta de información mediante una API REST.
- Proporcionar una interfaz gráfica que permita supervisar el estado del sistema.
- Mantener una arquitectura modular que facilite futuras ampliaciones.

B.3. Catálogo de requisitos

Requisitos funcionales

- **RF-01:** El sistema deberá procesar eventos procedentes de diferentes dispositivos simulados.
- **RF-02:** El sistema deberá almacenar los eventos en una base de datos.
- **RF-03:** El sistema deberá generar alertas mediante reglas de correlación.
- **RF-04:** El sistema deberá detectar anomalías sobre los eventos almacenados.
- **RF-05:** El sistema deberá proporcionar una API REST para la consulta de información.
- **RF-06:** El sistema deberá mostrar eventos y alertas mediante un dashboard web.
- **RF-07:** El sistema deberá mostrar el estado de los sensores monitorizados.
- **RF-08:** El sistema deberá permitir la visualización de estadísticas de funcionamiento.
- **RF-09:** El sistema deberá clasificar las anomalías según su nivel de riesgo.
- **RF-10:** El sistema deberá permitir la configuración de determinados parámetros operativos.

Requisitos no funcionales

- **RNF-01:** El sistema deberá ser modular y fácilmente mantenible.
- **RNF-02:** El sistema deberá utilizar tecnologías de código abierto.
- **RNF-03:** El sistema deberá ejecutarse con un consumo reducido de recursos.
- **RNF-04:** El sistema deberá ofrecer una interfaz sencilla e intuitiva.
- **RNF-05:** El sistema deberá facilitar futuras ampliaciones funcionales.

B.4. Especificación de requisitos

CU-01 Consultar dashboard CU-02 Consultar eventos CU-03 Consultar alertas CU-04 Consultar anomalías CU-05 Consultar estado de sensores CU-06 Configurar parámetros del sistema

CU-1	Ejemplo de caso de uso
Versión	1.0
Autor	Alumno
Requisitos asociados	RF-xx, RF-xx
Descripción	La descripción del CU
Precondición	Precondiciones (podría haber más de una)
Acciones	1. Pasos del CU 2. Pasos del CU (añadir tantos como sean necesarios)
Postcondición	Postcondiciones (podría haber más de una)
Excepciones	Excepciones
Importancia	Alta o Media o Baja...

Tabla B.1: CU-1 Nombre del caso de uso.

Apéndice C

Especificación de diseño

C.1. Introducción

En este anexo se describen los aspectos relacionados con el diseño del sistema desarrollado. Se detallan la estructura de los datos utilizados, la arquitectura general de la solución y los principales procesos implementados para el tratamiento de la información.

El objetivo es proporcionar una visión técnica de cómo se ha diseñado el sistema SIEM y de las decisiones adoptadas durante su desarrollo.

C.2. Diseño de datos

El sistema utiliza una base de datos SQLite para almacenar de forma persistente la información generada durante el funcionamiento del SIEM.

Las entidades principales utilizadas son los eventos y las alertas. Los eventos representan las acciones o sucesos detectados por los distintos dispositivos monitorizados, mientras que las alertas corresponden a situaciones generadas por el motor de correlación cuando se cumplen determinadas condiciones de seguridad.

La tabla de eventos almacena información relacionada con la fecha y hora del suceso, origen del evento, tipo de evento, severidad, usuario implicado, punto de acceso, depósito afectado, dispositivo generador, resultado de la operación y descripción del evento.

Por su parte, la tabla de alertas almacena la información generada por las reglas de correlación y los mecanismos de detección implementados en el sistema.

El diseño adoptado permite almacenar la información de forma sencilla y eficiente, facilitando las consultas posteriores realizadas por la API y el dashboard.

C.3. Diseño arquitectónico

La arquitectura del sistema se ha diseñado siguiendo un enfoque modular con el objetivo de facilitar el mantenimiento y la ampliación futura de la aplicación.

El sistema se encuentra dividido en varios componentes independientes que colaboran entre sí para procesar los eventos generados por el entorno simulado.

El módulo de ingesta es el encargado de leer y procesar los logs generados por los distintos dispositivos. Posteriormente, la información es almacenada en la base de datos SQLite y analizada por los módulos de correlación y detección de anomalías.

Una vez procesada la información, la API REST desarrollada con FastAPI proporciona acceso a los datos almacenados. Finalmente, el dashboard implementado mediante Streamlit permite visualizar eventos, alertas, anomalías y estadísticas del sistema.

Esta separación de responsabilidades permite desacoplar los distintos módulos y simplifica la incorporación de nuevas funcionalidades.

C.4. Diseño procedimental

El funcionamiento del sistema sigue una secuencia de procesamiento claramente definida.

En primer lugar, los dispositivos simulados generan eventos que son almacenados en archivos de logs. El módulo de ingesta procesa dichos eventos y verifica que cumplen el formato establecido.

Posteriormente, los eventos son almacenados en la base de datos y analizados por el motor de correlación y el sistema de detección de anomalías. Cuando se detecta una situación relevante, el sistema genera la correspondiente alerta.

Finalmente, la información almacenada puede ser consultada mediante la API REST y visualizada desde el dashboard de monitorización.

Este flujo de trabajo permite procesar de forma ordenada toda la información generada por el entorno monitorizado y facilita la supervisión continua del sistema.

Apéndice D

Documentación técnica de programación

D.1. Introducción

El presente anexo tiene como objetivo proporcionar la información técnica necesaria para comprender la estructura interna del proyecto y facilitar su mantenimiento o evolución futura.

Se describen los principales directorios del sistema, las instrucciones de instalación y ejecución, así como las pruebas realizadas para validar el correcto funcionamiento de la aplicación.

D.2. Estructura de directorios

El proyecto se encuentra organizado siguiendo una estructura modular que permite separar claramente las diferentes responsabilidades del sistema.

La estructura principal del proyecto es la siguiente:

- **api:** Contiene la implementación de la API REST desarrollada con FastAPI.
- **correlation:** Incluye el motor de correlación y las reglas utilizadas para generar alertas.
- **dashboard:** Contiene la aplicación web desarrollada con Streamlit.

- **db**: Gestiona el acceso a la base de datos SQLite y el almacenamiento de eventos y alertas.
- **ingestor**: Implementa el módulo encargado de procesar los logs generados por los dispositivos simulados.
- **machine**: Incluye los mecanismos de detección de anomalías.
- **logs**: Almacena los archivos de eventos utilizados por el sistema.
- **config**: Contiene los parámetros de configuración de la aplicación.
- **tests**: Directorio destinado a pruebas y validaciones del sistema.
- **main.py**: Punto de entrada principal de la aplicación.

D.3. Manual del programador

El desarrollo del proyecto se ha realizado utilizando Python como lenguaje principal de programación.

La arquitectura modular adoptada permite modificar o ampliar funcionalidades sin afectar significativamente al resto de componentes del sistema.

Para incorporar nuevas reglas de correlación, el desarrollador debe actuar sobre el módulo correspondiente dentro del directorio *correlation*. Del mismo modo, las nuevas reglas de detección de anomalías pueden añadirse dentro del módulo *machine*.

La API REST puede ampliarse incorporando nuevos endpoints dentro del módulo *api*, mientras que las nuevas funcionalidades de visualización pueden añadirse directamente en las páginas del dashboard desarrolladas con Streamlit.

Esta organización facilita el mantenimiento del código y simplifica la incorporación de futuras mejoras.

D.4. Compilación, instalación y ejecución del proyecto

```
python main.py python -m uvicorn api.server:app --reload python -m streamlit run dashboard/app.py
```

D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

E.1. Introducción

El presente anexo tiene como finalidad proporcionar la información necesaria para que un usuario pueda instalar, ejecutar y utilizar el sistema desarrollado.

Se describen los requisitos necesarios para su funcionamiento, el proceso de instalación y las principales funcionalidades disponibles a través de la interfaz de monitorización.

E.2. Requisitos de usuarios

Para utilizar el sistema es necesario disponer de un equipo con sistema operativo Windows, Linux o macOS que tenga instalado Python 3.10 o superior.

Asimismo, es necesario disponer de las dependencias utilizadas por el proyecto, principalmente FastAPI, Streamlit y las bibliotecas auxiliares incluidas en el fichero de requisitos.

Los requisitos hardware son reducidos, ya que el sistema ha sido diseñado como un SIEM ligero. El proyecto puede ejecutarse en equipos de características modestas e incluso en plataformas de bajo consumo como Raspberry Pi.

También se requiere un navegador web moderno para acceder al dashboard de monitorización y a la documentación de la API.

E.3. Instalación

Para instalar el proyecto es necesario descargar el código fuente desde el repositorio y situarse en el directorio principal del proyecto.

Una vez descargado el proyecto, se recomienda crear un entorno virtual de Python e instalar las dependencias necesarias.

Tras la instalación de las dependencias, el sistema puede ponerse en funcionamiento ejecutando el núcleo principal del SIEM, la API REST y el dashboard de monitorización.

La aplicación web estará disponible a través del navegador y permitirá consultar toda la información generada por el sistema.

E.4. Manual del usuario

El acceso al sistema se realiza mediante el dashboard web desarrollado con Streamlit.

La página principal muestra el estado general del sistema, estadísticas de funcionamiento, anomalías detectadas y actividad reciente. Esta información permite al usuario obtener una visión global del estado de la instalación monitorizada.

La página de eventos y alertas permite consultar los registros almacenados por el sistema. El usuario puede utilizar los filtros disponibles para localizar información concreta y analizar la actividad registrada.

La página de sensores y configuración muestra el estado de los dispositivos monitorizados y permite consultar determinados parámetros de funcionamiento del sistema.

Las anomalías detectadas aparecen clasificadas según su nivel de riesgo, facilitando la identificación de aquellas situaciones que requieren una atención prioritaria.

El dashboard incorpora además diferentes gráficos y estadísticas que permiten visualizar de forma rápida la evolución de los eventos y alertas generados por el sistema.

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

La sostenibilidad suele asociarse principalmente a aspectos medioambientales, aunque en realidad también incluye dimensiones sociales, económicas y tecnológicas. Durante el desarrollo de este Trabajo Fin de Grado he podido comprobar que muchas de las decisiones adoptadas en un proyecto tecnológico pueden contribuir de forma directa o indirecta a una utilización más eficiente y responsable de los recursos disponibles.

El proyecto desarrollado consiste en un sistema SIEM ligero orientado a la monitorización de eventos de seguridad dentro de un entorno simulado de alta seguridad. Aunque a primera vista pueda parecer un proyecto centrado exclusivamente en aspectos técnicos, durante su desarrollo se han aplicado diversos principios relacionados con la sostenibilidad.

Uno de los aspectos más relevantes ha sido la utilización de tecnologías de código abierto. Herramientas como Python, SQLite, FastAPI, Streamlit, Git o GitHub permiten desarrollar soluciones funcionales sin necesidad de adquirir licencias de software comerciales. Esto favorece la accesibilidad tecnológica y facilita que pequeñas organizaciones o instituciones educativas puedan disponer de herramientas avanzadas sin realizar grandes inversiones económicas.

Otro aspecto importante es la eficiencia en el uso de los recursos hardware. Desde el inicio del proyecto se planteó el desarrollo de un SIEM ligero capaz de ejecutarse en equipos con recursos limitados, incluyendo plataformas como Raspberry Pi. Este enfoque permite reducir el consumo energético y prolongar la vida útil de los equipos utilizados, contribuyendo a un uso más sostenible de la tecnología.

La sostenibilidad también está relacionada con la seguridad y la continuidad de los servicios. Los sistemas de monitorización permiten detectar incidencias, fallos o situaciones anómalas antes de que se conviertan en problemas de mayor gravedad. En este sentido, el proyecto contribuye a mejorar la supervisión de infraestructuras críticas mediante mecanismos de correlación de eventos y detección de anomalías que facilitan la identificación temprana de posibles riesgos.

Desde el punto de vista social, el desarrollo del proyecto ha permitido adquirir competencias relacionadas con el trabajo autónomo, la resolución de problemas y el aprendizaje continuo. A lo largo del desarrollo ha sido necesario analizar diferentes alternativas tecnológicas, tomar decisiones de diseño y adaptarse a nuevas herramientas y metodologías. Estas capacidades resultan fundamentales para el ejercicio profesional responsable dentro del ámbito de la ingeniería informática.

Asimismo, la utilización de herramientas de inteligencia artificial como apoyo al desarrollo ha permitido mejorar la productividad y agilizar determinadas tareas técnicas y documentales. No obstante, durante el proyecto se ha mantenido una actitud crítica respecto a la información generada por estas herramientas, revisando y validando siempre los resultados obtenidos antes de incorporarlos al sistema.

Por último, considero que este trabajo me ha permitido comprender mejor la importancia de diseñar soluciones tecnológicas eficientes, mantenibles y escalables. La sostenibilidad no debe entenderse únicamente como una cuestión medioambiental, sino también como la capacidad de desarrollar sistemas que aprovechen adecuadamente los recursos disponibles, puedan

mantenerse a largo plazo y aporten valor real a las organizaciones y a la sociedad.

En conclusión, el desarrollo de este Trabajo Fin de Grado ha contribuido a reforzar mi comprensión de los principios de sostenibilidad aplicados al ámbito tecnológico, mostrando cómo decisiones relacionadas con la arquitectura del sistema, las herramientas utilizadas y la gestión de los recursos pueden tener un impacto positivo desde el punto de vista económico, social y medioambiental.